

DiRAC GPPT Workshop

2026-09-15

DiRAC GPPT Workshop

Welcome!

<https://tinyurl.com/GPPTprogramme>

We will be recording voice!

The recordings will not be made public

This is for future training development

About today

<https://tinyurl.com/GPPTprogramme>

- Intended to give you experience in advanced GPU topics
- Very hands on
- Mostly focused on performance, profiling and optimisation
- Sprinkled in deep dives into real codes

Your friendly RSEs

- Dr Duncan Leggat (Fortran, C++, OpenMP, CUDA, AMD)
- Dr Miren Radia (Fortran, C++, OpenACC, CUDA, AMD)
- Dr Kacper Kornet (Everything? But especially tooling & CSD3)
- Dr Simon Clifford (Fortran, OpenACC, OpenMP)
- Me! (C++, CUDA, Python)
- Prof Simon Hands (Classical tenor)

Programming GPUs

Landscape & recap

This talk

- Why use GPUs?
- GPU hardware & software landscape
- Basic GPU programming
- GPU hardware & performance
- Then into exercises!

Who am I?

Dr Jamie Quinn

- Astrophysical & geophysical fluid dynamics
- Senior RSE @ UCL, ICL & DiRAC
- Now developing DiRAC's GPU training
- Specialise in CUDA C++, Python+CuPy, algorithms & software architecture

Why use GPUs?

	AMD EPYC 9965	Nvidia H100
"Cores"	192	114 (SMs)
FP64/cycle/core	16	64
GHz	2.25	1.1
TFLOPS	7	8-25*
RAM (GB/s)	500	2000

So why use GPUs?

- 4x Bandwidth
- 3x FLOPS (maybe)
- Tensor cores (extra x2 FLOPS)

What about 100x?

The CPU code was probably under-optimised.

Porting is an opportunity to reimplement / refactor / optimise

... and GPUs are not forgiving; non-optimal code is *slow*.

Warning: Porting may cause fully vectorised, parallelised, cache-efficient CPU code

GPU landscape

The players

Nvidia

- Recent: Blackwell
- CUDA ecosystem
 - More robust
 - Bugs can live for a long time
 - More performant
 - More funding
 - Runs on *all* consumer hardware
- Focus on AI

AMD

- Recent: MI3XX
- ROCm ecosystem
 - Open source
 - Lagging in features
- Hardware better suited to HPC
- Cheaper

GPU landscape

Trends

Stagnating FP64

- Hopper -> Blackwell *decreased* FP64 performance
 - 33 TFLOPS -> 30 TFLOPS
 - rebalancing FLOPS - MEMOPS
 - replaced with non-IEEE emulation
- AMD GPUs still improving FP64 performance
 - MI300X: 81.7 TFLOPS

Most FLOPs in a GPU is now in tensor cores

- Both AMD & Nvidia tensor hardware offer FLOPS twice that of regular OPs
- Tricky to use effectively
- Basically only GEMM
- But many libraries available

GPU memory is (slowly) increasing

- CPU systems easily host 2TB+ per node
- RAM per GPU is **80GB** (Hopper 2022) up to **288GB** (Rubin 2026)
 - Typically 4-8 GPUs per node, so ~2TB per node (for 20x the price...)

GPU landscape

Software

- Kernel-based (**CUDA**, HIP, OpenCL*)
- Directive-based (**OpenMP**, OpenACC*)
- Standard library parallelism(`std::par, do concurrent`)
- Frameworks (SYCL, Kokkos, Raja)
- Parallel languages (Futhark, Chapel, Mojo)
- Python JIT/Numpy (JAX, CuPy, Numba, PyTorch, Taichi)
- Tile-based** (cuTile, FlyDSL)
- Fortran is supported less and less

*avoid

**watch closely

How to choose?

All exist in a vector space of:

- usability
- optimisation potential
- hardware portability

Always a compromise...

Or is it?

Interoperability

GPU Landscape

Library ecosystems

Official:

- Nvidia: [CUDA-X Libraries](#)
- AMD GPU: [ROCm Libraries](#)
- AMD CPU: [AMD Optimizing CPU Libraries \(AOCL\)](#)
- Intel: [OneAPI](#)

Independent:

- PETSc
- Trilinos
- RAPIDS
- GSL

Break!

Questions?

Coming up... Basic GPU programming

What is a GPU?

A hardware view

- CPU cores -> *streaming multiprocessors*
- Each SM executes ~4 *warps*
- Each warp executes 32 *threads* in lockstep
- Threads execute *kernels*

1 warp instruction is analogous to a CPU vector instruction

- CPU core: **2 AVX-512 (2x8 FP64)**
- SM: **4 executing warps (4x16 FP64)**

GPUs are (kind of) wide vector machines!

But GPUs are *SIMT*, not *SIMD*.

SIMT vs SIMD

Single Instruction Multiple Data

- vector lane 0 accesses index 0
- vector lane 1 accesses index 1
- ...
- data pre-packed into vectors *before* execution

Single Instruction Multiple Threads

- thread i calculates its own data index

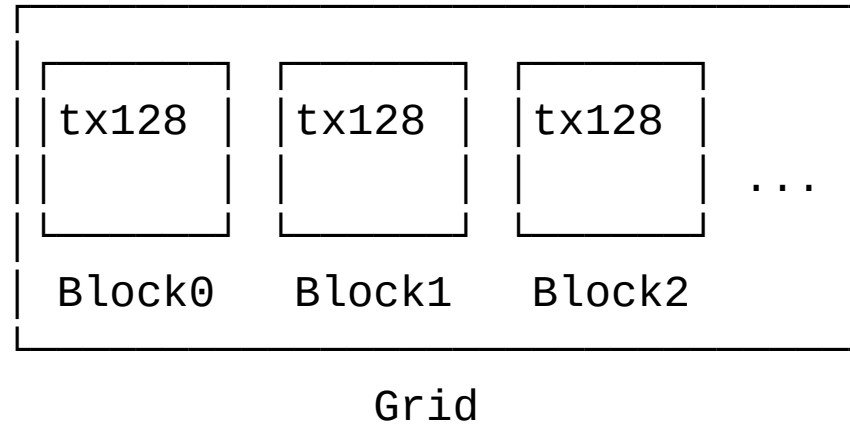
Difference is small. GPU kernels are slightly more flexible.

The memory hierarchy

- Host memory - O(1 TB)
- GPU memory - O(100 GB)
- L2 cache - O(100 MB)
- Shared memory / L1 cache O(10 MB)
- Registers O(1e4 registers) *per SM*

What is a GPU?

A software view



- A **kernel** launch creates a **grid** of **blocks**
- Each block is made of a set number of **threads**
- Each thread executes the kernel body

The blocks are split into warps!

Keep in mind

- Thread-level (one kernel body per thread)
- Block-level (relevant for `__syncthreads()`)
- Warp-level (this is the hardware's view!)

Terminology

Nvidia	AMD	OpenMP
warp	wavefront	N/A
thread	work unit	thread
block	workgroup	team
streaming multiprocessor	compute unit	N/A

Writing kernels: CUDA

```

1  for (int i = 0; i < n; i++) {
2      out[i] = a*x[i] + y[i];
3  }
4
5  __global__ void axpy(float *out, const float *x, const float *y, float a, int
6      int i = blockIdx.x * blockDim.x + threadIdx.x;
7      if (i < n) {
8          out[i] = a*x[i] + y[i];
9      }
10 }

```

- `__global__` signifies a *kernel*
- Pointers must be to GPU memory (unless managed)
- Constants copied automatically
- Thread calculates its own index
- Guard out-of-bounds (loop length != data length)
- “Loop body” remains similar

```
1  __global__ void axpy(float *out, const float *x, const float *y, float a, i
2
3  ...
4
5  cudaMalloc(d_out, ...);
6  cudaMalloc(d_x, ...);
7  cudaMemcpy(d_x, h_x, ...);
8
9  int n_blocks = (n + block_size - 1) / block_size; // 1 thread per item
10
11  axpy<<<n_blocks, block_size>>>(d_out, d_x, d_y, a, n);
12
13  cudaDeviceSynchronize();
14  cudaMemcpy(h_out, d_out, ...);
```

OpenMP

omp target

```
1 #pragma omp target teams distribute parallel for \  
2   map(to: x[0:n],y[0:n]) map(from: out[0:n])  
3 for (int i = 0; i < n; i++) {  
4   out[i] = a*x[i] + y[i];  
5 }  
6  
7 // synchronous by default!
```

- `target` — offload the region to the device
- `teams distribute` — split loop across teams/blocks
- `parallel for` — split each team across threads
- `map(to: ...)` — copy to device before
- `map(from: ...)` — copy from device after

omp target data

```
1 #pragma omp target data map(to: x[0:n],y[0:n]) map(from: out[0:n])
2 {
3 #pragma omp target teams distribute parallel do
4     for (int i = 0; i < n; i++) {
5         out[i] = a*x[i] + y[i];
6     }
7     ... // other GPU operations
8 }
```

What about Fortran?

- C++: `#pragma omp` operates on `{ }` scopes
- Fortran: `!$omp` sometimes needs `$!omp end`

```
1 - #pragma omp target data
2 - { ... }
3
4 + !$omp target data
5 + ...
6 + !$omp end target data
```

Building

```
1 nvcc -O3 -arch=native -lineinfo -g <file>
```

```
1 nvfortran -mp=gpu -Minfo=mp -g -O3 <file>
```


Break!

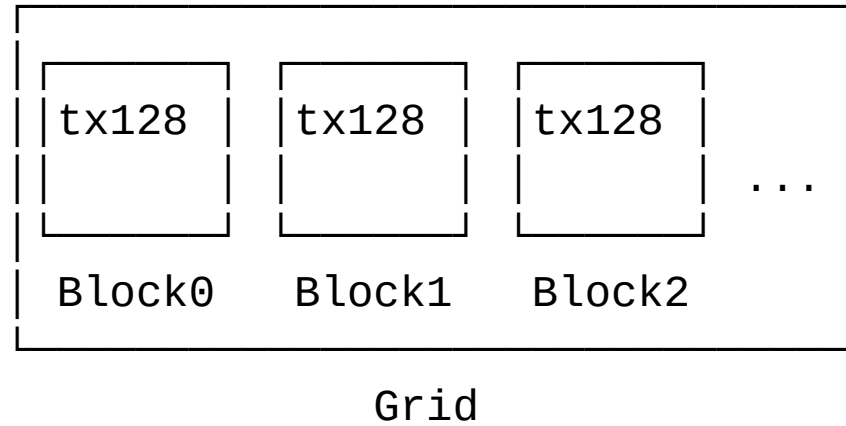
Questions? Back at 11:30!

Coming up... GPU performance

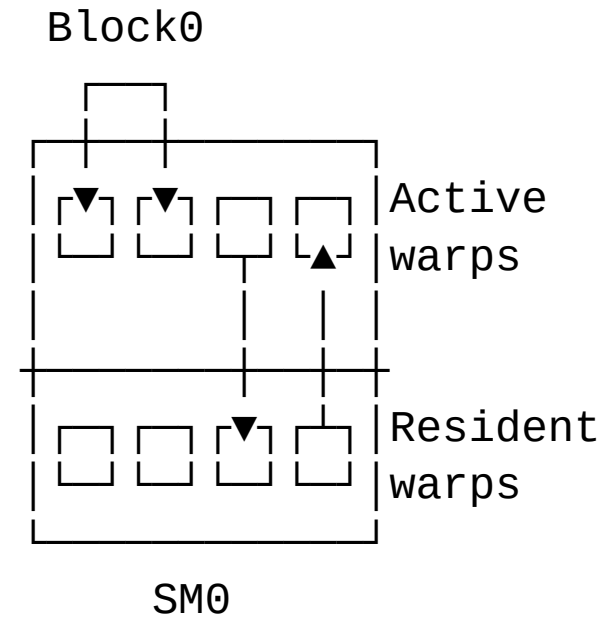
GPU Performance

... but first

What is a GPU? II



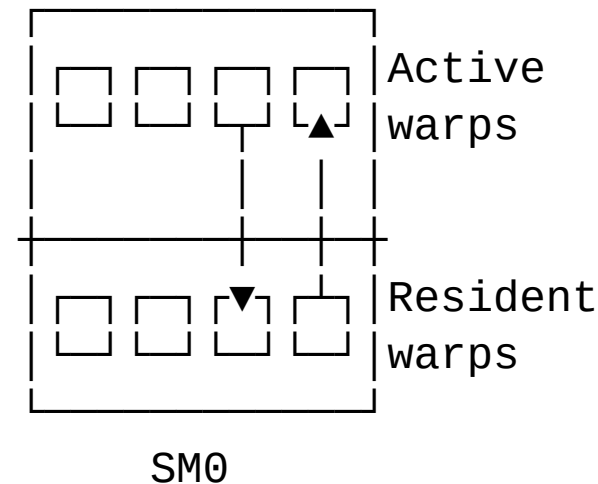
- A **kernel** launch creates a **grid** of **blocks**
- Each block is made of a set number of **threads**



- Each block is assigned to an SM
- The SM splits the block into **warps** of size 32
- Each warp executes all its threads
- The SM can quickly swap warps

The warp scheduler

Each SM has 4 schedulers, each with 4+ active warp slots:



When a warp stalls (e.g. from a memory access) the scheduler can switch *within a clock cycle*. This is key to general GPU performance.

Occupancy is the ratio of active / maximum warps.

Measures ability of GPU to *hide latency*.

High-latency instructions & memory access limit *actual* occupancy.

Limited resources like registers and shared memory limit *theoretical occupancy*.

H100 SMs have 4 schedulers with 64 active warps!

I.e. 2048 active threads and 128 currently executing.

Where should data live?

Ideal is:

```
1 cudaMemcpy(d_x, h_x, ...);  
2  
3 run_entire_simulation();  
4  
5 cudaDeviceSynchronize();  
6 cudaMemcpy(h_out, d_out, ...);
```

Why?

PCIe 5: ~60 GB/s

GPU RAM: 2000+ GB/s

How?

Wait

Latest AMD & Nvidia cards have O(200) GB / GPU.

Use streaming

Stream data to/from GPU as required. Key optimisation is interleaving compute & data transfers.

Go multi-GPU

Split problem across many GPUs. Key technologies include MPI, collective libs, SHMEM libs.

How do GPUs do branching?

Each warp executes *all* instructions but *masks* the output.

```
1  __global__ void branch(float *out, int n) {
2      int i = blockIdx.x * blockDim.x + threadIdx.x;
3      if(i >= n) return;
4
5      if(i%2 == 0) {
6          out[i] = 1.0;    // runs with mask 01010101
7      } else {
8          out[i] = sin(i); // runs with mask 10101010
9      }
10 }
```

Branches take as long as the *sum* of their runtimes.

How do GPUs access memory?

CPU & GPU memory hardware load data in *cache lines* (64-128 bytes).

On GPU, 1 warp (32 threads) *should* access contiguously for ideal memory performance.

Uncoalesced case:

```
1 __global__ void my_kernel(float* x) {  
2     int i = blockIdx.x * blockDim.x + threadIdx.x;  
3     x[32*i] += 1;  
4 }
```

0 -> +0 bytes

1 -> +32 * 4 bytes

2 -> +64 * 4 bytes

3 -> +96 * 4 bytes

...

Coalesced case:

```
1 __global__ void my_kernel(float* x) {  
2     int i = blockIdx.x * blockDim.x + threadIdx.x;  
3     x[i] += 1;  
4 }
```

0 -> +0 bytes

1 -> +4 bytes

2 -> +8 bytes

3 -> +12 bytes

...

31 -> +124 bytes

All threads in a warp can access 1 **FP32** each in one load!

How do GPUs organise kernels?

Streams

Launch GPU operations concurrently:

```
1 cudaMemcpy(..., stream1);  
2 axpy<<<n_blocks, threads_per_block, 0, stream1>>>(d_out, d_x, d_y, a, n);  
3  
4 cudaMemcpy(..., stream2);  
5 axpy<<<n_blocks, threads_per_block, 0, stream2>>>(d_out, d_x, d_y, a, n);
```

- Increase occupancy
- Overlap data transfers & comms

Shared memory & `__syncthreads()`

```
1  __global__ void axpy(float *out, const float *x, const float *y, float a, i
2  int tid = threadIdx.x;
3  int gtid = blockIdx.x * blockDim.x + threadIdx.x;
4
5  __shared__ int sdata[THREADS_PER_BLOCK];
6
7  // Load data from global to shared memory
8  if(gtid < n) sdata[tid] = y[gtid];
9  __syncthreads(); // all threads in *block* must reach here
10
11 // Do something with shared memory
12 if(gtid < n) sdata[tid] += a*x[gtid];
13 __syncthreads();
14
15 // Store result back to global memory
16 if(gtid < n) out[gtid] = sdata[tid];
17 }
```

Note: `__syncthreads()` operates on *block* level, not warp level.

We can get shared memory in OpenMP with *team-private* arrays (see exercise tomorrow).

Beware **bank conflicts** (not covered here).

Misc GPU features

Host/device functions

```
1  __device__ float calc_axpy(float x, float y, float a) {
2      return a * x + y;
3  }
4
5  __global__ void axpy(float *out, const float *x, const float *y, float a, i
6      int i = blockIdx.x * blockDim.x + threadIdx.x;
7      if (i < n) {
8          out[i] = calc_axpy(x[i], y[i], a);
9      }
10 }
```

omp target declare

```
1 // __device__ float calc_axpy(float x, float y, float a) {  
2 //   return a * x + y;  
3 // }  
4  
5 #pragma omp declare target  
6 float calc_axpy(float x, float y, float a) {  
7   return a * x + y;  
8 }
```

Testing

```
1  __device__ __host__ float calc_axpy(float x, float y, float a) {
2      return a * x + y;
3  }
4
5  __global__ void axpy(float *out, const float *x, const float *y, float a, int n) {
6      int i = blockIdx.x * blockDim.x + threadIdx.x;
7      if (i < n) {
8          out[i] = calc_axpy(x[i], y[i], a); // runs on GPU
9      }
10 }
11
12 ...
13
14 test_calc_axpy() {
15     calc_axpy(...) // runs on host!
16 }
```

Thrust: Ergonomic data transfer

```
1  std::vector<float> x_h(n);
2  thrust::device_vector<float> x_d(n);
3  thrust::copy(x_h.begin(), x_h.end(), x_d.begin());
4
5  axpy<<<n_blocks, threads_per_block>>>(d_out.data().get(), ...);
6
7  cudaDeviceSynchronize();
8  thrust::copy(out_d.begin(), out_d.end(), out_h.begin());
```

Also available in AMD flavours

Performance recommendations

- Set `block_size` to a multiple of 32 to fit warps
- Track occupancy
- Avoid data transfers (& use thrust)
- Avoid branching
- Ensure coalesced memory accesses

Coffee & Exercises

Back @ 11:30

<https://tinyurl.com/GPPTprogramme>

Lunch!

Back @ 14:00